

Shipboard Offline Document Q&A Progress Report

You handed me 9 manuals — 1,335 pages — on 31 August. This is what I built in the month to 29 September, why I built it that way, where the evidence sits, and what comes next.

Repository branch `ship-rag`, HEAD = `de1188a`. Every number and every code reference in this deck points at that commit, so you can open it and check as we go.

What I built this month is foundation — and it rebuilds byte-for-byte

Three artefacts are frozen: the corpus, the evaluation yardstick, and model admission. Each carries a sha256, two independent runs produce byte-identical output, and anyone who clones the repo gets the same copy.

On your question — whether to fine-tune — I did not answer it directly. I turned it into a controlled experiment **I can lose**. Its prerequisite was a trustworthy yardstick, and that closed on 28 September.

Next is the retrieval baseline — the first time this project produces experimental numbers. The retrieval layer, generation layer and shipboard service are still stubs; I set that out on page 5.

3,409

corpus chunks · c8978777

38 / 38

citations resolved at L1 or L2

158

tests — on the layers that exist

3

decisions I need from you (p.18)

I use one test to decide what to do first: **make errors die early**

How far an error travels between being created and being caught is what determines its cost. An error first caught on the vessel costs the most, because nobody out there can fix it. So what I optimised this month is not any one model — it is the path an error travels.

A model that can't load on the vessel

dies at gate 2 of selection, not after the experiment is finished and its lead model turns out to be unshippable.

A safety margin that is actually negative

dies on paper during the arithmetic check, not as a timeout at sea.

A chunk with the right shape and garbled content

dies during content spot-checks, not when a crew member is shown a garbled citation.

A wrong inference of my own

dies the moment I write it down, not after it has hardened into an architecture decision.

That test also sets the order of this report: first how I make errors die early, then progress. The next dozen pages are all instances of it.

Fine-tuning: I made it an experiment, not an answer

I do have a prior, and I wrote it down

I lean towards retrieval mattering more, because the core requirement is "with a citation", and fine-tuning does not produce citations. **But that is a prior, not a conclusion.** I may be wrong.

	No retrieval	Retrieval
No fine-tune	A — base model	B — RAG only
Fine-tune	C — fine-tune only	D — RAG + fine-tune

The four arms are defined in core/contracts.py:377 and recorded per result row for attribution.

The decision rule is strict on me too

A paired test looks only at discordant pairs, not at the net difference. Sampling is off everywhere (:239) — at n=31, a few questions flipping by sampling alone move the result 6 percentage points.

If fine-tuning wins, I still won't take it at face value

If arm C does well, we need a held-out set built by excluding the source chunks at training time, to separate **real generalisation** from **memorising the training corpus**. That is already in the BACKLOG — not something I thought of when asked.

Why it hasn't run yet: all four arms share one yardstick. The gold-chunk map only froze on 28 September; any run before that produces numbers that cannot go into the same table.

My plan is a dependency chain, and I'm at the entrance to link three

Why this order: each link needs the one before

1 · Citations trustworthy

If citations aren't trustworthy, nothing can be checked and the product premise fails

2 · Yardstick trustworthy

Only then can a human-written quote be mapped onto machine-made chunks

3 · Parameters comparable

With a yardstick in hand, all four arms still need the same budget

4 · Conclusions comparable ← the 2×2 sits here

If any earlier link is loose, the four arms' numbers can't be compared at all

Execution sequence and exit criteria

Step	Exit criterion	Status
S0–S3 environment / contract	Contract and test set frozen, hashed	Done
S4a primary model ready	Gates 1–2 measured, model selected	Done
S5 · S5b · S5c corpus	9/9 parsed, two runs byte-identical	Done · 3,409 chunks
S6 gold-chunk map	Every citation L1/L2, two runs identical	Done · in Git
S4a.9 budget calibration	All four load-bearing constants measured	Open · 9d missing
S8 retrieval baseline	Three retrievers + four recall levels + per-question detail	Next
S9 · S10 calibration, pre-registration	Threshold from score distribution; pre-reg hashed	Not started
S11 · S12 fine-tune and four arms	156 result rows on disk + blind grading	Not started

Against the milestones: M0–M1b closed, currently M1c, then M2 controlled experiment → M3 cross-model evaluation → M4 own codebase → M5 tuning → M6 shipboard → M7 single-vessel pilot. ⚠️ The "hallucination <5%, citation accuracy >90%" figures in the M5 criteria are a **future acceptance bar, not today's status** — I do not have those two numbers.

The citation is load-bearing structure, not decoration

I define this system as an **offline, citation-bearing document navigator**. It turns a crew member's question into "which passage of which manual"; the sentence it generates is a summary of that evidence, not the answer itself.

No citation, no answer

Enforced in code, not suggested in a prompt.

"Not in the manuals" is a valid return

Refusal is not an exception.
Answer(refused=True) at contracts.py:593 is a normal state.

Citations survive a dead generator

On timeout it degrades to retrieval only, and the crew can still follow the reference.

Hence the shore / vessel split

Parsing, chunking, indexing and fine-tuning all happen on shore. The vessel only does retrieve → generate → show the citation. One reason only: **every extra component on the vessel is one more failure point nobody out there can repair**. There is no IT support on board, so we don't even install OCR there.

So the citation is frozen as a triple, and parsing splits by page

The triple `contracts.py:472`

A citation is (manual id, section, PDF page). The printed page number is stored separately for a human to cross-check — each chapter restarts its footer at 1, so one manual contains many "page 1". **It cannot be used to locate anything.**

Split by page `kaiva_pdf.py:111`

Page boundaries align closely with section boundaries. **Merging across pages would give one chunk two sections, and the triple would immediately stop being unique.** An over-long page is split further within the page, and the pieces share all of that page's metadata.

Where the section comes from: three levels, in order

Level	Location	Rule
1	:296	SECTION label in the page header, first occurrence wins
2	:427	No label — take the centred title line, extract the number or the whole line
3	parse()	Still empty — inherit from the previous page

Two hard rules

Never return an empty list — an empty list means only "genuinely nothing retrieved", never "it crashed". If pdfinfo and pdftotext disagree on page count, raise — silent truncation is the hardest failure to find.

The parser's class docstring states what it deliberately **does not** do: no OCR, and no judgement about whether the text layer is readable. It pushed that responsibility out explicitly — the next page is about the fact that nothing caught it.

So the citation gate must be independent of body quality — "9" is the counterexample

Every check was green

9/9 manuals parsed, 3,383 chunks, normal length distribution, 100% section coverage, two builds byte-identical.

But the content was garbage

66 pages of TACM embed a font with no Unicode mapping, so extraction yields junk characters. Those pages left 50 chunks, all of which pass the four checks above.

And my own prediction was refuted

I expected garbled pages to inherit the previous section. Measured: of 50 chunks, **50 came from their own page and 0 inherited** — level 2 returned non-empty junk, so level 3 never fired.

On one of those pages the section field resolved to "9" — and TACM really does have a chapter 9. Non-empty, well-formed, a member of the known section set: three checks passed, semantics entirely unrelated.

This is not a parser bug — its docs say it does not judge text-layer readability. The problem is that **no second component was catching what it had pushed out**. That counterexample now lives in the module docstring of `citation_gate.py`.

So the gate answers one question `citation_gate.py:161`

Can the triple reliably support a citation. **It does not judge whether the body text is good**. Candidates from both channels enter; no source is preferred a priori.

Five prohibitions

Two candidates that disagree mean uncertain. No auto-pick, no choosing by known vocabulary, no neighbouring-page override, no inheritance, no voting.

So the citation decides admission — dropping 81 pages is deliberate

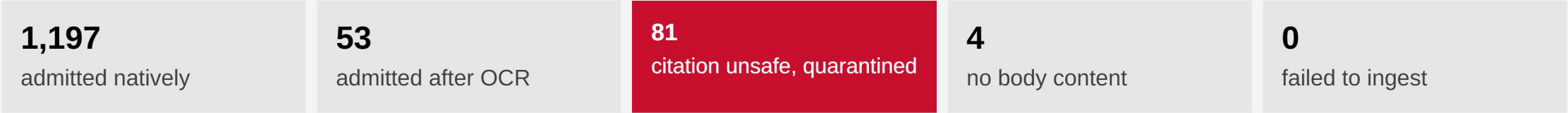
Gate result	Native channel	OCR channel
explicit_supported	Admit	Admit
uncertain	Admit + audit flag	Quarantine
failed	Quarantine	Quarantine

The **asymmetry is deliberate**. The native channel is the existing baseline; tightening it would shift the corpus identity and invalidate every earlier measurement. OCR is the channel introduced this round, and **a new channel has to be stricter**. Implemented at [phase_b_ingest.py:213–247](#), which reads states only, never raw signals.

Why I accept losing 81 pages

If we can't find it, the crew opens the paper manual. If we hand them a plausible-looking wrong citation, they act on the wrong page. **The two failures do not cost the same**. All 81 pages carry full attribution in the quality table and can be handled by category at any time; the cost will be quantified by the per-question detail in S8.

All 1,335 pages are accounted for, and the totals close



140 OCR artefacts are frozen in Git; 138 were actually consumed. CMM pages 32 and 112 have artefacts but their native text was already readable, so they take the native path. "An artefact exists" must never be an input to "this page should use OCR" — that would be circular.

Only now can I build the yardstick — and it must not live with the questions

The problem

The questions are written by a human, and each citation says "this page, this sentence". Retrieval works in chunks. To score retrieval at all, I first have to know which chunks each answer lands in.

Why it is not inside the test set

Questions and citations are **human judgement** — lose them and they cannot be regenerated. The map is **machine-derived from one version of the corpus**; change the corpus and it must be recomputed. Different lifetimes — in one file, someone eventually edits one while updating the other.

Structure	Line	Nature
EvalItem	:618	Excludes gold_chunk_ids
GoldChunkMap	:663	Separate derivative, 7 identity fields

It is an exception that went into Git

My default rule is "anything one command can regenerate stays out of Git". This table is an exception, with seven reasons written down. The core one: **every later result cites it as the gold provenance, so it must be recoverable from a fresh clone** — I cannot require each new environment to re-run S6 first. The size (about 20 KB) is explicitly recorded as **not** a reason.

The rule: unique minimum cover, or the whole map is void

Defined at `contracts.py:428`, implemented at `resolve_gold_chunks.py:126 / :169`

Level	Criterion	Measured
L1	Minimum cover unique, and one chunk suffices	30
L2	Minimum cover unique, needs two or more chunks	8
AMBIGUOUS	A cover exists but the minimum cover is not unique — no tie-break at all	0
L3	No full cover, but an individually-matchable fragment hits	0
L4	Page exists, no cover, no eligible hit → the quote was transcribed loosely	0
FAIL	That page has no chunk in the corpus → page missing	0

Fail-closed

If any citation is not L1/L2, no accepted map is produced at all — only the detail report, and exit code 3.

Three shortcuts I forbade myself

Look only at the cited page, **never neighbouring pages**; a non-unique minimum cover must be marked uncertain, **never picked by order** (that would add a preference the citation never gave); **L4 and FAIL must stay separate**, or a mis-transcribed quote disguises itself as a missing page.

Section is not a matching precondition

"Chapter 15" versus "15" must not create false negatives; it only feeds the audit. Three mismatches measured — all still matched on the same page.

How I show the yardstick is trustworthy, not merely "it ran"

1 · Two runs agree

Two independent processes, two fresh directories, one committed command — map and detail report are **byte-identical**.

2 · Independent audit

Without importing the implementation, recompute **all** minimum covers from the contract: 0 violations, 0 ties, candidate space per page ≤ 3 .

3 · 99 tests

Including an object-identity test: the builder identity handed to the validator must be the authoritative module **itself** — an equal-valued forgery does not count.

4 · Synthetic vectors + mutation

Four levels are 0 in real data, so synthetic cases pin them down; then I broke the reference implementation on purpose to confirm the matching case really fails.

5 · I removed a gate of my own that could never fail

The old form computed the corpus hash on the spot and then passed that value to the program to verify against — **the expected value and the object under test came from the same source, so any file whatsoever would pass**. That is a structurally unfalsifiable gate; it is the same as no gate. The expected value now comes only from the frozen record, and the locally computed one is used purely for comparison. **And I fixed only that one operational gate — I did not expand it into a new contract.**

I also wrote down the limit: the Git tree alone **cannot** re-prove that every chunk in the map exists in the corpus, because the corpus is not in Git. That layer uses the on-disk corpus as an explicitly labelled read-only external input. That is not a failure, it is the boundary of the verification scope — and it must not be described as "the whole acceptance replays from Git alone".

Beyond the yardstick, all four arms need one budget — every load-bearing constant needs provenance

The contract splits thresholds into two sections and **the physical separation is deliberate**: calibrated ones at :87, uncalibrated ones at :253, so misuse is visible at a glance.

Constant	Value	Line	Provenance
MAX_PROMPT_TOKENS	1050	:143	Four-point bench + fine sweep at target
PROMPT_OVERHEAD_RESERVE_TOKENS	200	:317	Estimate, uncalibrated
CONTEXT_PACK_MARGIN	0.90	:195	Fails the safety inequality
CONTEXT_PACK_BUDGET_TOKENS	765	:321	Sole definition of the derived value
CHARS_PER_TOKEN_EST	4	:190	No tokenizer dependency; tail measured
CITATION_HEADER_EST_TOKENS	14	:201	Measured p50 10 / max 28, error both ways

Three stages

First subtract the prompt overhead, which can be measured exactly; **then** apply the estimation margin to what remains. The two steps cannot be merged — one is measurable, the other can only be given headroom.

One scoping error, with a concrete cost

The old name MAX_CONTEXT_TOKENS hid the fact that this is the budget for **the whole prompt**. The packer gave all of it to chunks; the system prompt and citation headers were free riders — measured 12.4 s, 24% over budget. The comment records this as the **third** instance of "a number with a source but an unchecked scope".

Two parameters fail. I found them, and I did not change them yet

The margin fails on arithmetic alone

$$0.90 \times 1.15 = 1.035$$

It claims to absorb $\pm 15\%$ estimation error, yet the product exceeds 1. The correct upper bound is $1 \div 1.15 = 0.8696$, rounded down to **0.86**. Not 0.87 — $0.87 \times 1.15 = 1.0005$ still overruns, and at certain overhead values integer truncation lets it **pass the self-check by luck**.

The second one: the 200-token overhead reserve is an **estimate**; the measured candidate is 206. It decides whether S8 reports recall at rank 2 or rank 3.

Why I did not change them immediately

A measurement is evidence; changing the contract is a decision. They must sit in different rounds and different commits. Mixed together, it later becomes impossible to say which number rests on which evidence. The measurement commit of 25 September was done exactly that way — not one line of contract change in it.

And the contract admits it breaks the line

$765 \times 1.15 + 200 = 1080 \rightarrow 10.13 \text{ s} > 10.0$. The comment immediately adds: "**do not nudge the margin to 0.88 to get under the line — that is moving a parameter to make a number look good.**"

Re-measured on the new corpus

Dangerous tail of the estimation error: **110 / 3,409**. Simulated over real windows, the current settings overrun in **35 / 3,405** cases, by at most 338 tokens. Boundary: 338 tokens is roughly 3.5 s on the dev machine, and **that conversion does not extrapolate to the vessel**.

I put the gates before performance, so the faster model did not win

The ledger holds 7 entries with role: candidate. Five are obtained and measured (below); granite4:tiny and smollm3 are **not yet obtained**. **Three passed gate 2, across 3 families**. The M3 cross-model evaluation itself has not started.

Candidate	Params	Gate 1	Gate 2	Note — gates 3 (licence) and 4 (multilingual) are all TODO
phi4-mini	3.84B	PASS	PASS	Selected as the M2 primary; pp512 107.47 / pp1024 98.15 t/s
llama3.2:3b	3.21B	PASS	PASS	Faster, and 5/5 on the language test — but its licence is self-flagged AMBER, so gate 3 keeps it out of the comparison
qwen3:4b	4.02B	PASS	PASS	Language smoke 1/5, marked NOT_CLEAN, not cited as a clean measurement
gemma3:4b	4.3B	PASS	FAIL	key not found: gemma3.attention.layer_norm_rms_epsilon
qwen3.5:4b	4.7B	PASS	FAIL	rope.dimension_sections array length 3, expected 4

Why gates come first

One candidate ran perfectly well in another environment, but the inference engine we will use on the vessel failed to load it outright. Find that last and every earlier experiment has an unshippable lead.

Failures carry a scope too

A FAIL binds only to "this model file plus this runtime build" — never generalised into architecture incompatibility. Both GGUF magic numbers are fine; full diagnostics stored separately.

Three independent reasons for the primary

Wider budget, cleaner licence, no separate thinking mode that could be switched on by mistake. The 5/5 language smoke is supporting evidence only, never load-bearing on its own.

Seven things that are not finished — my own list

Item	Status and impact
Budget calibration open	The re-measurement on the new corpus is committed on its own; step 9d and the contract change ritual are not done, so the step is still marked OPEN
18f556e	The commit message says "calibrate / correct contracts", but that commit changes 0 lines of core/contracts.py — the title describes the intent; only the evidence actually landed
contracts.py:148–177	The chunk-length figures in the comments (1169 / 1491 / 1893) and CD01's "3 chunks" are still from the old corpus; measured today it is 1097 / 1390 / 1515 and CD01 is 5 chunks. Conclusions unchanged; to be corrected together with 9d
MIN_RELEVANCE = 0.35	Uncalibrated, with a known validity risk — calibrating on the same 39 questions I then use to test refusal is tuning on the test set. The mitigation is leave-one-out, to quantify the optimism as a number
Multi-gold hit semantics	Recorded as a downstream contract gap, not frozen. Must be decided before S8 starts
Gates 3 and 4	Licence and multilingual gates are TODO for every candidate, the primary included. The licence is currently only "a claim seen at one source on one date"
Execution manual S8.3	The command example still computes the hash locally and compares it to itself — the same shape as the gate I killed on page 12. First thing to fix before S8 starts

This page is not a disclaimer. All seven are already written down in the repo — decision log, execution-manual status blocks, contract comments, ledger TODO fields. I collect them here because what usually goes wrong in a report is not "got it wrong" but "didn't mention it".

What I deliver next, how it is accepted, and one prediction written in advance

Acceptance checklist for the S8 retrieval baseline

- Three retrievers (keyword / vector / hybrid), **none importing another**
- Relevance normalised to 0..1 inside the component before it leaves; raw scores never escape
- Recall at ranks 1 / 2 / 3 / 20 for each; **trap questions excluded from the denominator**
- Cross-document questions scored per manual, not collapsed to "any gold chunk"
- **The three multilingual questions reported separately** — English corpus, non-English query, so this is cross-lingual retrieval; if it is weak, that is the embedding model, not the generator. **This attribution has to be settled before the four-arm experiment**
- Per-question detail plus ten failure tags; the fusion method written as a module-level constant

Reranker decision rule, fixed in advance

Recall@2 $\approx$ Recall@20 $\rightarrow$ don't add it; the latency and failure surface saved are the gain. @20 high but @2 low $\rightarrow$ worth trying. @20 itself low $\rightarrow$ the problem is recall, not ordering. **If hybrid is worse than a single route, I still may not tune the fusion weights to fix it** — that is tuning on the test set.

This is not writing product code early

Components depend only on the contract file and never on each other, so swapping a model is a config change, not a code change. This code is an asset for the M4 in-house codebase; none of it is wasted.

One unfavourable prediction I am writing down now: CD01's gold is 5 chunks across 2 manuals, while a 765-token budget fits 2 chunks at the median — **it may be structurally unanswerable**. This goes into the "most likely outcome" field of the pre-registration. A prediction made in advance is evidence; an explanation made afterwards is an excuse.

Three things still undecided

1 · What counts as a hit when a question has several gold chunks — needed before S8 starts

Ten questions in the map have more than one gold chunk; the largest, CD01, has 5 across 2 manuals. Any-one-hits / all-must-hit / proportional give very different scores, and under a budget that fits 1–2 chunks, "all must hit" pins that question at 0 forever. **My preference: any-one-hits as the headline metric, all-must-hit as a supplementary observation, and report both.** Rule first, code second — choosing the algorithm afterwards is no different from tuning on the test set.

2 · The vessel hardware spec — it governs every latency conclusion

Every speed figure today is the dev machine on CPU only (a positive control proves the GPU is not in play, though Accelerate and unified memory bandwidth still are). The historical estimate for vessel x86 is 35–85 t/s, so 1,050 tokens is **12.35 s ($1050 \div 85$) to 30 s ($1050 \div 35$)**, against a 10 s budget. **That tension cannot be closed before real hardware arrives, so the latency contract stays provisional.**

3 · Whether to freeze the budget contract before S8

The evidence is complete; what is missing is step 9d and one contract change ritual (pages 14 and 16). **My recommendation is to complete it first:** otherwise whether S8 reports recall at rank 2 or rank 3 rests on a margin known to be inconsistent and an uncalibrated overhead reserve. The cost is about one day of delay to S8.

Default path: if you have no objection to these three, I proceed with budget contract close-out → retrieval baseline → threshold calibration → pre-registration → four-arm experiment.